RF Track Write-up

RF-H1: Two Radios, One Channel

1. What I tried first, and why it did not work

My first idea was to open the IQ recording in a spectrum analyzer and use a frequency filter to isolate the weak signal. That could not work, because the loud signal and the hidden weak signal were on the same frequency and used the same bandwidth. Any filter that removed the loud signal would have removed the weak one too.

2. The insight that actually cracked it

I used Successive Interference Cancellation (SIC) to subtract the loud signal. I first demodulated the loud BPSK signal to recover its bits and timing. From those I generated a clean, noiseless reconstruction of the loud waveform. Since I was working with raw I/Q data, I could subtract that reconstruction directly from the original recording. With the loud signal removed, the noise floor dropped and the weak BPSK signal underneath became visible.

3. How I satisfied myself the answer was right

Before the subtraction, the weak signal was not visible at all and looked like static. After the subtraction, its constellation diagram resolved into two tight clusters at +1 and -1, which is what clean BPSK looks like. Running a standard demodulator on it gave a readable ASCII flag.

4. Where I used an AI assistant and what I had to correct

I had not heard of Successive Interference Cancellation before. I asked an AI how to separate two signals on the same frequency, and it explained the idea of subtracting the loud waveform. I understood the concept, but lining up the phase of the reconstructed signal with the original recording in Python was difficult. The AI helped me write a fine-sync function to align the arrays before the subtraction.

RF-H2: Chipped

1. What I tried first, and why it did not work

A standard BPSK demodulator run directly on the recording would not work. The signal was Direct-Sequence Spread Spectrum (DSSS) and sat below the noise floor. Without the spreading code, a normal demodulator would only output random bits.

2. The insight that actually cracked it

The symbol length was 127 chips, so the spreading code repeated every 127 chips. That meant I could try to recover the code from the data itself using linear algebra. I split the recording into blocks of 127 chips and stacked them into a large matrix. Squaring the blocks removes the +1/-1 data modulation, so what remains is the repeating chip structure. I ran Principal Component Analysis (PCA) on this matrix, and the strongest principal component turned out to be the spreading code.

3. How I satisfied myself the answer was right

I cross-correlated the code from the PCA step against the raw recording and got strong, regularly spaced peaks standing out of the noise. Peaks like that only appear if the code matches the one actually used in the transmission. With the code confirmed, I pulled out the bits and read the flag.

4. Where I used an AI assistant and what I had to correct

I did not know how to recover a DSSS spreading code. I asked an AI for help, and it suggested splitting the signal into blocks and using PCA to find the repeating structure. I had never implemented PCA, so the AI gave me the sklearn.decomposition snippet. I had to reshape the NumPy arrays myself to fit what it expected, but the PCA logic itself worked as given.

RF-H3: Fingerprint

1. What I tried first, and why it did not work

I first thought about identifying the rogue transmitter from software settings such as baud rate and center frequency. That was a dead end, because the rogue operator had copied all of those settings from another radio on the network, which made that radio look guilty.

2. The insight that actually cracked it

Software settings can be copied, but hardware imperfections cannot. An analog mixer always leaks a small image of the signal on the opposite side of the center frequency because of hardware imbalance. I used an FFT to measure the power difference between the main signal and this image, which is the Image Rejection Ratio (IRR), for all 8 reference radios. I then measured the IRR of the rogue transmission, and it matched the hardware profile of a different radio from the one it was imitating.

3. How I satisfied myself the answer was right

The IRRs of the reference radios were clearly different from each other, spread over several decibels. The rogue transmission claimed to be Radio X, which had an IRR of about 45 dB, but the rogue's measured IRR was about 32 dB, which matched the true culprit's radio. Since the hardware imbalance is not something the operator could copy through settings, I was confident in the match.

4. Where I used an AI assistant and what I had to correct

I used an AI to help write the script that finds the peaks in the FFT. I knew I had to measure the main peak and the image peak, but locating them in the FFT bins by code was tedious. The AI gave me a short snippet using scipy.signal.find_peaks, and I adjusted it to search for the image peak on the mirrored side of the spectrum.

RF-H5: Ghost Carrier

1. What I tried first, and why it did not work

I had found the secret DSSS GPS C/A codes, but the loud 16-channel FSK frequency-hopping signal was interfering with the correlation. I considered blanking (zeroing) the time slices where the FSK bursts occurred. That would not work, because zeroing those slices also removes the DSSS chips underneath, and correlation would fail.

2. The insight that actually cracked it

Instead of blanking in the time domain, I filtered in the frequency domain. I applied narrow notch filters at the specific frequencies where the FSK bursts hopped. DSSS is very wideband, so notching a few narrow frequencies removed most of the FSK interference and left enough of the DSSS signal to work with. That let me correlate the GPS codes and track the phase.

3. How I satisfied myself the answer was right

After the notch filtering, I ran a PLL (Phase-Locked Loop) to track the signal, and it locked onto the phase. The decoded bits were 3 Shamir Secret Shares. Those only combine into readable text if all 3 are correct, and combining them gave the decrypted flag, which confirmed the pipeline was working.

4. Where I used an AI assistant and what I had to correct

I had little practical experience with DSP filtering. When blanking in the time domain failed, I asked an AI for alternatives, and it suggested frequency-domain notch filters. Designing the poles and zeros for a digital IIR filter by hand was beyond me, so I had the AI give me the scipy.signal.iirnotch function to generate the coefficients. I then applied it at the frequencies I had mapped from the FSK hops.